# Lab 1: Smart Contract Basics & Ethereum Mechanism Exploration

## 1. Objectives

1. Master the Development Environment
   Become familiar with Remix IDE(https://remix.ethereum.org/), a browser-based development environment for writing, compiling, and deploying smart contracts.
2. Understand Solidity Fundamentals
   Learn the basic structure of a Solidity contract, including License declaration, Pragma version, and Contract definition.
3. Understand Ethereum's Account Model
   Explore the two types of accounts in Ethereum:
   • Externally Owned Accounts (EOA)
   • Contract Accounts
4. Explore the Economic Model of Blockchain Operations
   Understand the distinction between:
   • State-changing transactions (which consume Gas)
   • Read-only calls (which can be executed locally without Gas)

# 2. Background: Decentralized Tip Jar

Traditional donation boxes often suffer from low transparency. Funds collected may be misused by administrators, and donors cannot verify how the funds are handled.

Blockchain technology provides a transparent and verifiable ledger. In this experiment, you will implement a decentralized donation contract (Tip Jar) that ensures:

• Transparency – Anyone can inspect the contract balance and donation records on the blockchain.
• Security through Code Logic – Funds can only be handled according to predefined rules written in the smart contract.

This experiment demonstrates how programmatic rules can enforce trust in decentralized systems.

# 3. Requirements

## 3.1 Basic Functionalities (Mandatory)

Write and deploy a smart contract named TipJar that includes the following functionalities.

### 3.1.1. Admin Mechanism

Use the variable

address public owner;

to store the address of the contract creator.

The owner should be initialized in the constructor using:

owner = msg.sender;

### 3.1.2. Donation Interface

Implement a payable function named donate() which allows users to send ETH to the contract.

You may use the following built-in variables:

• msg.sender – the address of the caller
• msg.value – the amount of ETH sent with the transaction

### 3.1.3. Balance Inquiry

Implement a view function that returns the current ETH balance stored in the contract.

For example:

```
function getBalance() public view returns (uint) {
return address(this).balance;
}
```

### 3.1.4. Fund Withdrawal

Implement a function withdraw() so that only the contract owner can withdraw the funds.

Use require() to enforce access control. For example:

require(msg.sender == owner, "Only owner can withdraw");

# 3.2 Advanced Features (Mandatory)

Extend your contract by implementing the following mechanisms.

## 3.2.1. Blacklist Mechanism

Maintain a mapping:

mapping(address => bool) public blacklist;

The admin should be able to mark addresses as blacklisted.
Blacklisted addresses must not be allowed to donate.

Example logic:

require(!blacklist[msg.sender], "Address is blacklisted");

## 3.2.2. Crowdfunding Goal

Define a constant goal amount:

uint public constant GOAL = 5 ether;

The donate() function should stop accepting donations once the contract balance reaches this goal.

Example logic:

require(address(this).balance < GOAL, "Funding goal reached");

## 3.2.3. Data Analytics

Track the following statistics within the contract:

• Highest Single Donation Amount
• Total Number of Unique Donors

Possible variables include:

uint public highestDonation;
uint public donorCount;

mapping(address => bool) public hasDonated;

A donor should only be counted once, even if they donate multiple times.

### 3.2.4. Time Lock (Cooling Period)

Implement a time restriction so that the owner can only perform the first withdrawal 5 minutes after contract deployment.

Use the following variable:

uint public deployTime;

Initialize it during deployment:

deployTime = block.timestamp;

Withdrawal should only be allowed after:

block.timestamp >= deployTime + 5 minutes

# 4. Operation Guide & Tips

## 4.1 Understanding Ethereum Account Types

During the experiment, pay attention to the following two types of addresses.

**Externally Owned Accounts (EOA):** These are the addresses shown in the ACCOUNT dropdown menu in Remix.

Characteristics:

- Controlled by private keys
- Used by users to send transactions
- Can interact with smart contracts

**Contract Accounts:** After deploying the contract, an address will appear in the Deployed Contracts section.

Characteristics:

- Controlled entirely by smart contract code
- No private key
- Executes predefined logic automatically

A contract account can be viewed as a programmable autonomous system.

## 4.2 Key Solidity Concepts

The following variables and functions are commonly used in this lab.

msg.sender – address of the caller
msg.value – amount of ETH sent with the transaction
address(this).balance – ETH stored in the contract
require() – condition check that reverts if false
block.timestamp – timestamp of the current block

# 5. Suggested Steps

**Step 1 — Write the Contract**
Create a new .sol file in Remix IDE and implement the required contract logic.
Ensure all warnings and syntax errors are resolved.

**Step 2 — Compile**
Open the Solidity Compiler panel and select a suitable compiler version (for example 0.8.20).
Compile the contract until the green checkmark appears.

**Step 3 — Deploy**
Open the Deploy & Run Transactions panel.
Set the environment to Remix VM and click Deploy.
After deployment, the contract instance will appear under Deployed Contracts.

**Step 4** — Interact and Test

Perform the following tests:

- Switch between different ACCOUNT addresses
- Send donations from different accounts

- Test the blacklist restriction
- Attempt to call withdraw() as both the owner and a non-owner
- Observe the transaction logs and contract state changes

# 6. Assessment Requirements

Submit a Lab Report (PDF format, no template) including the following components.

## 6.1. Source Code

Provide the complete Solidity source code of your final contract.

## 6.2. Deployment Screenshot

Include a screenshot showing:

- the Deployed Contracts panel
- the contract address

## 6.3. Functional Verification

Include screenshots demonstrating the following:

1. Blacklist mechanism working
   Show the transaction failure message when a blacklisted address attempts to donate.
2. Contract balance increasing
   Show the contract balance increasing as donations are made.

**Submission Instructions:** Please submit your final report in PDF format to the **"Assignments -> Lab-1"** section of the Learningcom (Xuexitong) module for this course.

**Deadline: March 15th, 23:59:59** (The submission portal is now open).

# 实验一：智能合约基础与以太坊机制初探

## 1. 实验目标

1. **掌握开发环境**
   熟悉 Remix IDE（https://remix.ethereum.org/），这是一个用于编写、编译和部署智能合约的基于浏览器的集成开发环境。
2. **掌握 Solidity 基础**
   学习 Solidity 合约的基本结构，包括代码版权声明（License）、编译器版本（Pragma）和合约定义（Contract）。
3. **理解以太坊账户模型**
   探索以太坊中的两类账户：
   - 外部账户 (Externally Owned Accounts, EOA)
   - 合约账户 (Contract Accounts)
4. **探索区块链操作的经济模型**
   理解以下两者之间的区别：
   - 改变状态的交易（消耗 Gas）
   - 只读调用（在本地执行，不消耗 Gas）

## 2. 实验背景：去中心化捐款箱

传统的捐款箱往往透明度较低。收集到的资金可能被管理员挪用，且捐赠者无法验证资金的具体去向。 区块链技术提供了一个透明且可验证的账本。在本实验中，你将实现一个去中心化的捐赠合约（Tip Jar），确保：

- **透明性** – 任何人都可以检查区块链上的合约余额和捐赠记录。
- **通过代码逻辑实现安全性** – 资金只能根据智能合约中预定义的规则进行处理。 本实验展示了如何通过程序化规则在去中心化系统中建立信任。

## 3. 实验要求

### 3.1 基础功能（必做）

编写并部署一个名为 TipJar 的智能合约，包含以下功能：

#### 3.1.1. 管理员机制

使用变量 address public owner; 存储合约创建者的地址。 owner 应在构造函数中初始化：owner = msg.sender;

#### 3.1.2. 捐赠接口

实现一个 payable 函数 donate()，允许用户向合约发送 ETH。 你可以使用以下内置变量：

- msg.sender – 调用者的地址
- msg.value – 随交易发送的 ETH 数量

**3.1.3. 余额查询** 实现一个 view 函数，返回合约中存储的当前 ETH 余额。例如：

Solidity
```
function getBalance() public view returns (uint) {
    return address(this).balance;
}
```

**3.1.4. 资金提取**

实现函数 withdraw()，确保只有合约所有者可以提取资金。 使用 require() 强制执行访问控制。例如： require(msg.sender == owner, "Only owner can withdraw");

## 3.2 进阶功能（必做）

通过实现以下机制扩展你的合约：

**3.2.1. 黑名单机制**

维护一个映射：mapping(address => bool) public blacklist; 管理员应能将特定地址标记为黑名单，黑名单地址不允许进行捐赠。 示例逻辑：
require(!blacklist[msg.sender], "Address is blacklisted");

**3.2.2. 众筹目标**

定义一个常量目标金额：uint public constant GOAL = 5 ether; 一旦合约余额达到此目标，donate() 函数应停止接收捐赠。 示例逻辑：
require(address(this).balance < GOAL, "Funding goal reached");

**3.2.3. 数据分析**

在合约中跟踪以下统计数据：

- 单笔最高捐赠金额
- 独立捐赠者总数 建议使用的变量： uint public highestDonation; uint public donorCount; mapping(address => bool) public hasDonated; 即使一个捐赠者多次捐赠，也只能计算一次。

#### 3.2.4. 时间锁（冷却期）

实施时间限制，使得所有者只能在合约部署 **5 分钟后** 进行第一次提取。 使用变量：uint public deployTime; 在部署期间初始化：deployTime = block.timestamp; 仅在以下条件满足时允许提取：block.timestamp >= deployTime + 5 minutes;

# 4. 操作指南与提示

## 4.1 理解以太坊账户类型

在实验过程中，请注意以下两类地址：

**外部账户 (EOA)**：Remix 面板 ACCOUNT 下拉菜单中显示的地址。

- 特点：由私钥控制；用户用于发送交易；可以与智能合约交互。

**合约账户**：部署合约后，地址将出现在 Deployed Contracts 区域。

- 特点：完全由智能合约代码控制；没有私钥；自动执行预定义的逻辑。

## 4.2 核心 Solidity 概念

- msg.sender – 调用者地址
- msg.value – 随交易发送的 ETH 数量（单位为 Wei）
- address(this).balance – 合约中存储的 ETH
- require() – 条件检查，如果为 false 则回滚交易
- block.timestamp – 当前区块的时间戳

# 5. 建议步骤

1. **编写合约**：在 Remix IDE 中新建 .sol 文件并实现要求的逻辑。确保解决所有警告和语法错误。
2. **编译**：打开 Solidity 编译器面板，选择合适的版本（如 0.8.20），进行编译直到出现绿色对勾。
3. **部署**：打开部署面板，环境设置为 Remix VM，点击 Deploy。
4. **交互与测试**：
    - 切换不同的 ACCOUNT 地址进行测试。
    - 从不同账户发送捐赠。
    - 测试黑名单限制。
    - 尝试以所有者和非所有者身份调用 withdraw()。
    - 观察控制台日志和合约状态变化。

# 6. 实验考核要求

提交一份实验报告（PDF 格式，无模版），中英文不限，包含以下部分：

## 6.1. 源代码

提供最终合约的完整 Solidity 源代码。

## 6.2. 部署截图

包含显示以下内容的截图：

- Deployed Contracts 面板
- 合约地址

## 6.3. 功能验证

包含展示以下内容的截图：

1. **黑名单机制生效**：展示当黑名单地址尝试捐赠时的交易失败消息。
2. **合约余额增加**：展示随着捐赠进行，合约余额增加的过程。

**最终报告以 pdf 的形式，提交到学习通本课程模块下-作业-Lab-1（已发布，有效期至 3.15 - 23:59:59）.**